A

MINI PROJECT REPORT

ON

"LINGUISTIC BRIDGE: BRIDGING COMMUNICATION GAPS THROUGH GENERATIVE AI AND MULTIMODAL DOCUMENT PROCESSING"

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGNCE & MACHINE LEARING

Submitted by

G.Rahul (23R91A7360)

Under The Guidance Of

Mrs.J.SWATHY

Assistant Professor



Department of Artificial Intelligence & Machine Learning

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(An Autonomous Institution)

Medbowli, Meerpet, Saroornagar, Hyderabad – 500097

(Affiliated to JNTUH, Approved by AICTE, Accredited by NBA & NAAC)

(2026)

CERTIFICATE

This is to certify that the Mini Project report on **"LINGUISTIC BRIDGE: BRIDGING COMMUNICATION GAPS THROUGH GENERATIVE AI AND MULTIMODAL DOCUMENT PROCESSING"** is a bonafide work carried out by **G.Rahul** (23R91A7360) in partial fulfillment for their requirement of the award of **Bachelor Of Technology In Artificial Intelligence & Machine Learning, Teegala Krishna Reddy Engineering College, Hyderabad**, affiliated to **Jawaharlal Nehru Technological University, Hyderabad** under my guidance and supervision.

The result of investigation enclosed in this report have been verified and found satisfactory. The results embodied in the project work have not been submitted to any other University for the award of any degree.

PROJECT CO ORDINATOR

Mrs.J. Swathy

Assistant professor

HEAD OF THE DEPARTMENT

Dr. Vadivelan Natrajan

Professor

EXTERNAL EXAMINER

PRINCIPAL

Dr.K.Venkata Murali Mohan

Professor

DECLARATION

We hereby declare that the Mini Project report entitled "**AI-DIRVEN HEALTHCARE PREDICTION SYSTEM WITH PERSONALISED RECOMMENDATIONS AND DOCTOR LOCATOR**" is done under the guidance of **J. Swathy** (Assistant Professor)**, Department of Artificial Intelligence & Machine Learning,** Teegala Krishna Reddy Engineering College**,** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** Artificial Intelligence & Machine Learning from **Jawaharlal Nehru Technological University, Hyderabad.**

This is a record of bonafide work carried out by us in **Teegala Krishna Reddy Engineering College** and the results embodied in this project have not been reproduced or copied from any source.

Submitted by

G.Rahul (23R91A7360)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowned our efforts with success.

We extend my deep sense of gratitude to **Dr. K. Venkata Murali Mohan**, **Principal** Teegala Krishna Reddy Engineering College, Meerpet, for permitting me to undertake this project. We are indebted to **Dr. Vadivelan Natrajan, Professor & Head of the Department**, Artificial Intelligence and Machine Learning, Teegala Krishna Reddy Engineering College, Meerpet for his support and guidance throughout our project.

We are indebted to the project coordinator **J. Swathy** Assistant professor, Artificial Intelligence and Machine Learning, Teegala Krishna Reddy Engineering College, Meerpet for his support and guidance throughout our project.

Finally, we express thanks to one and all that have helped me in successfully completing this mini project. Further I would like to thank my family and friends for their moral support and encouragement.

By

G.Rahul ((23R91A7360)

ABSTRACT

In the contemporary information-driven society, the complexity of technical, legal, and medical documentation has created a significant "accessibility gap." While digital transformation has made information more available, the linguistic sophistication required to comprehend these documents often excludes a large portion of the population, including students, non-native speakers, and individuals with cognitive disabilities. This project, titled **"Linguistic Bridge,"** presents an innovative solution to this crisis by leveraging state-of-the-art Generative Artificial Intelligence (AI) to automate the simplification of complex documents.The core of the system is built upon the **Gemini 2.5 Flash** large language model (LLM) architecture, which facilitates high-fidelity semantic analysis and text regeneration. Unlike traditional "rule-based" or "statistical" simplification methods that often destroy context, the Linguistic Bridge utilizes **Transformer-based Neural Networks** to understand the underlying intent of a document. It performs "Semantic Refactoring" to break down nested sentences, remove archaic jargon, and lower the reading grade level to a universal 6th-grade standard while strictly preserving the original factual integrity of the source material.A key technical highlight of this research is its **Multimodal Document Processing** capability. Developed using the **Flask (WSGI)** framework, the application integrates advanced document parsing libraries such as **PyMuPDF (fitz)** and **python-docx**. This allows the system to accept not only raw text but also binary file formats including **PDF** and **Word Documents**. The backend pipeline manages a secure stream-based extraction process, cleans the resulting text, and executes sophisticated prompt engineering to guide the AI toward an accessibility-focused output.The frontend is designed with a **User-Centric (UX) focus**, featuring a side-by-side comparative grid that allows users to verify the simplified text against the original source in real-time. Performance metrics indicate that the system significantly reduces cognitive load and reading time. By democratizing access to complex information, the Linguistic Bridge serves as a vital tool for digital inclusivity, ensuring that essential knowledge is no longer a barrier but a shared resource for all.

CONTENTS

	TITLE	PAGE NO
1.	**INTRODUCTION**	**1-2**
2.	**LITERATURE SURVEY**	**3-5**
3.	**ARCHITECTURE**	**6-11**
4.	**MODULES**	**12-13**
5.	**SOFTWARE REQUIREMENTS**	**14-15**
6.	**HARDWARE REQUIREMENTS**	**16-17**
7.	**SAMPLE CODE**	**18-29**
8.	**OUTPUT SCREENS**	**30-32**
9.	**CONCLUSION**	**33-35**
10.	**FUTURE ENHANCEMENTS**	**36-38**
11.	**REFERENCES**	**39-40**

LIST OF FIGURES

Fig no:3.1	Visual System Representation	7
Fig no:3.3	Use Case Diagram	7
Fig no:3.4	State Diagram	8
Fig no:3.5	Sequence Diagram	9
Fig no:3.6	Package Diagram	9
Fig no:3.7	Component Diagram	10
Fig no:3.8	Communication Diagram	10
Fig no 3.9	Composite Diagram	11
Fig no: 8.1	Output Screen 1	31
Fig no:8.2	Output Screen 2	31
Fig no:8.3	Output Screen 3	32

List Of Tables

Table 2.4	Market Survey	05

1.INTRODUCTION

1.INTRODUCTION

1.1 Project Overview and Philosophical Context

In the modern digital age, information is widely available due to the rapid growth of technology and the internet. This phenomenon is often described as the **democratization of knowledge**, where people can easily access large amounts of information online. However, despite this accessibility, a major challenge still exists — the **Linguistic Divide**. Many documents such as legal agreements, medical instructions, research papers, and technical manuals are written in complex language that is difficult for the general public to understand.The **Simple-Read system** is designed to address this problem by acting as an AI-based bridge between experts who produce complex information and everyday readers who need to understand it. The system uses the **Gemini 2.5 Flash large language model** to simplify complicated text while preserving its original meaning.This process can be described as **Semantic Refactoring**, where the structure and wording of the text are modified without changing the underlying facts. By simplifying vocabulary, breaking down long sentences, and improving clarity, the system ensures that important information becomes easier to understand for readers with different educational backgrounds.

The main philosophy behind Simple-Read is that **true access to knowledge depends not only on availability but also on understanding**. By reducing linguistic complexity, the system helps make information more inclusive and accessible to everyone.

1.2 Motivation: The Accessibility Gap

The motivation for developing Simple-Read comes from the **global accessibility gap in written information**. Although digital platforms provide access to large amounts of content, many people still struggle to understand complex documents.Studies in global literacy indicate that **nearly 50% of adults have difficulty understanding text written at a high school level or higher**. As a result, individuals may find it difficult to interpret legal documents, medical prescriptions, government policies, or academic materials. This situation creates **Information Exclusion**, where people technically have access to information but cannot fully understand it.

For example, patients may misunderstand medical instructions, citizens may struggle to interpret legal agreements, and students may find academic materials overly complex. These challenges can lead to confusion and limit individuals' ability to make informed decisions.

Simple-Read aims to solve this problem by automatically converting complex text into **Plain English at approximately a 6th-grade reading level**. This helps users understand important information quickly and clearly while maintaining the original meaning of the text. In this way, the system promotes **digital inclusivity and knowledge accessibility**.

1.3 Detailed Scope and Boundaries

The **Simple-Read platform** is designed as an accessibility tool that simplifies complex text into understandable language. The system integrates artificial intelligence, natural language processing, and user-friendly design to support effective communication.One key feature of the system is **Multi-Format Text Ingestion**, which allows users to input text through typing, copy-paste, or file uploads. This flexibility enables the system to process various types of documents, including reports, articles, and technical materials.Another important feature is **Semantic Preservation**, which ensures that the simplified text maintains the same meaning and factual accuracy as the original content. The system analyzes the context and structure of sentences before generating a simplified version.The platform also supports **High-Volume Processing**, allowing it to handle long documents efficiently using the optimized architecture of Gemini 2.5 Flash.To improve user experience, the system provides a **side-by-side comparison interface**, where the original text and the simplified version are displayed together. This helps users understand the transformation process and learn from it.

While the system focuses on improving readability, it currently works mainly with **English-language text** and does not perform full translation across multiple languages.

2.LITERATURE SURVEY

2.LITERATURE SURVEY

2.1 Historical Perspectives on Text Simplification (1990s–2010s)

Early research in Automated Text Simplification (ATS) was purely **Rule-Based**. Systems like "SYSTRAN" used hard-coded linguistic rules to shorten sentences. However, these rules often failed to account for the nuance of human language. In the 2010s, **Statistical Machine Translation (SMT)** became the standard, where algorithms looked for statistical correlations between complex and simple word pairs. While an improvement, these systems often produced "robotic" and unnatural results.

2.2 The Rise of Transformer Models (2017–Present)

The introduction of the "Transformer" architecture by Google revolutionized NLP. By using "Attention Mechanisms," models began to understand the relationship between words across long distances in a sentence. This project leverages the latest iteration of this lineage—**Gemini 2.5**. Unlike early BERT or GPT-2 models, Gemini 2.5 is "instruction-tuned," meaning it specifically understands the command to "simplify" while maintaining a strict adherence to the source text's truth.

2.3 Gap Analysis: Current Market Solutions

A thorough review of existing tools (Hemingway, Grammarly, DeepL) revealed a significant gap. Most tools provide "Suggestions" for better writing but do not perform "Automatic Rewriting" for a target demographic (e.g., 6th graders). Furthermore, few tools offer a unified pipeline that combines **OCR/Text Extraction** from PDFs with **Generative Simplification**. Simple-Read fills this void by providing an end-to-end, file-to-simple-text pipeline.

2.4 Advances in Large Language Models for Text Understanding

Recent developments in **Large Language Models (LLMs)** have significantly improved the ability of machines to understand and generate human-like text. Models such as **BERT, GPT, and Gemini** have demonstrated strong performance across a wide range of Natural Language Processing (NLP) tasks including summarization, translation, question answering, and text simplification.

Unlike earlier statistical models, modern LLMs are trained on massive datasets consisting of books, articles, websites, and academic papers. Through this large-scale training process, these models learn complex linguistic patterns, contextual relationships, and semantic structures within language. As a result, they are capable of producing text that is more natural, coherent, and contextually accurate.

Another important advancement is **instruction tuning**, where models are trained to follow specific user instructions. This enables systems to perform specialized tasks such as summarizing information, rewriting content, or simplifying complex sentences. In the context of text

simplification, instruction-tuned models can interpret commands like “explain in simple terms” or “rewrite for beginners” while preserving the original meaning of the text.

2.5 Importance of AI-Based Accessibility Tools

Accessibility has become a major focus in modern software development, especially in educational and informational platforms. Many individuals face barriers when interacting with complex textual content due to limited literacy levels, language differences, or cognitive challenges. AI-based accessibility tools aim to reduce these barriers and ensure that information can be understood by a wider audience.

Text simplification systems are particularly valuable in fields such as **education, healthcare, legal services, and public administration**, where clear communication is essential. For example, simplifying medical instructions can help patients better understand their treatment plans, while simplifying government policies can improve civic participation and awareness.

Recent research highlights the importance of designing AI systems that prioritize **clarity, transparency, and inclusivity**. By converting complicated information into plain and structured language, AI-based tools empower users to make informed decisions and engage more effectively with digital content.

The **Simple-Read platform** contributes to this growing field by combining modern AI models with a user-friendly interface that simplifies complex documents automatically. By focusing on readability and accessibility, the system helps bridge the gap between technical knowledge and everyday understanding, making information more inclusive for diverse groups of users.

Tool / System	Main Purpose	Features	Limitations
Hemingway Editor	Improve readability and writing style	Highlights complex sentences, suggests shorter alternatives, checks readability score	Only provides suggestions; does not automatically rewrite full text
Grammarly	Grammar and writing correction	Grammar checking, tone detection, vocabulary suggestions	Focuses on writing improvement rather than simplifying text for beginners
DeepL Translator	Language translation	High-quality translation across multiple languages	Primarily designed for translation, not for simplifying complex content
Simple-Read (Proposed System)	Automatic text simplification for accessibility	AI-based rewriting, converts complex text to plain language, side-by-side comparison interface, file-to-simplified-text pipeline	Currently focuses mainly on English text and simplification tasks

Table **2.4 Market Survey**

3. ARCHITECTURE

3.ARCHITECTURE

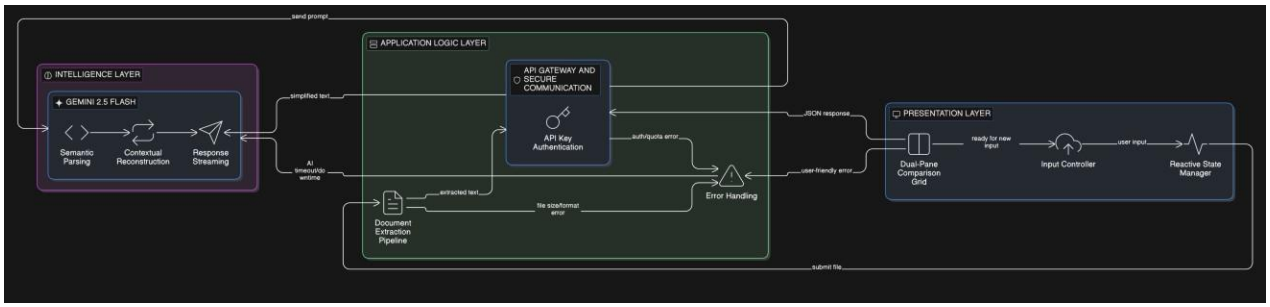


Fig no:3.1 Visual System Representation

3.1 Visual System Representation

To understand the interaction between the Flask server, the document parsers, and the Gemini API, we utilize several architectural views.

3.2 System Architecture Diagram

This diagram illustrates the "Big Picture" of the application. It shows how the user's device (Client) communicates with your Python server (Backend), which in turn acts as a gateway to the Google AI Cloud.

- **Tier 1 (Presentation):** The HTML/CSS/JS interface where the user interacts.
- **Tier 2 (Application):** The Flask server handling file uploads and text extraction.
- **Tier 3 (Inference):** The Gemini 2.5 Flash model providing the simplified text.

3.3 Use Case Diagram

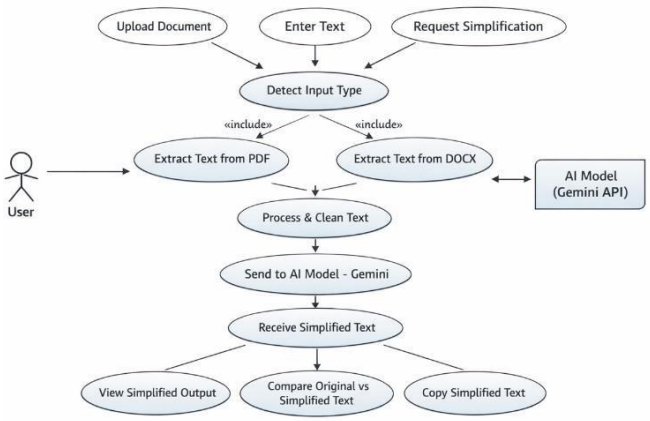


Fig no:3.3 Use Case Diagram

The use case diagram represents how the user interacts with the Linguistic Bridge (Simple-Read) system. The user can upload a document or enter text and request simplification. The system processes the input, extracts text if needed, and sends it to the AI model for simplification. Finally, the user can view, compare, and copy the simplified output.

3.4 State Diagram

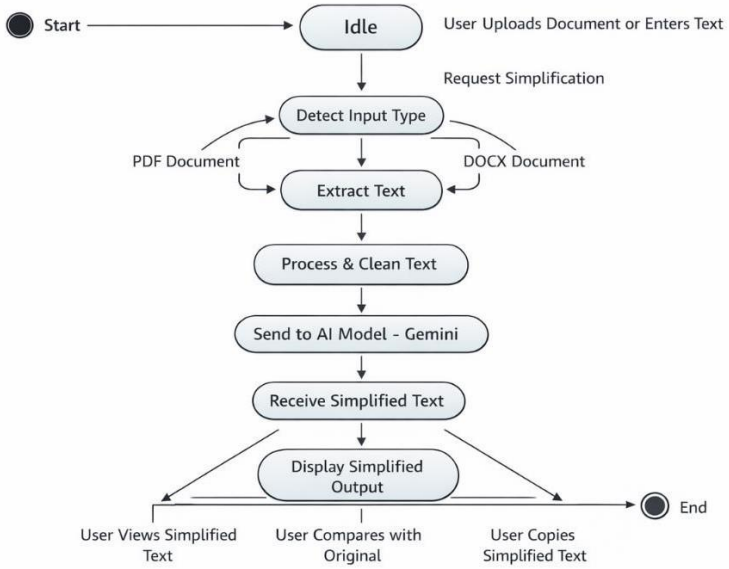


Fig no 3.4 State Diagram

The state diagram shows the flow of the system from start to end. It begins in the idle state where the user uploads a document or enters text. The system then detects the input type, extracts and processes the text, and sends it to the AI model for simplification. After receiving the simplified text, the system displays the output, allowing the user to view, compare, or copy it before reaching the end state.

3.5 Sequence Diagram

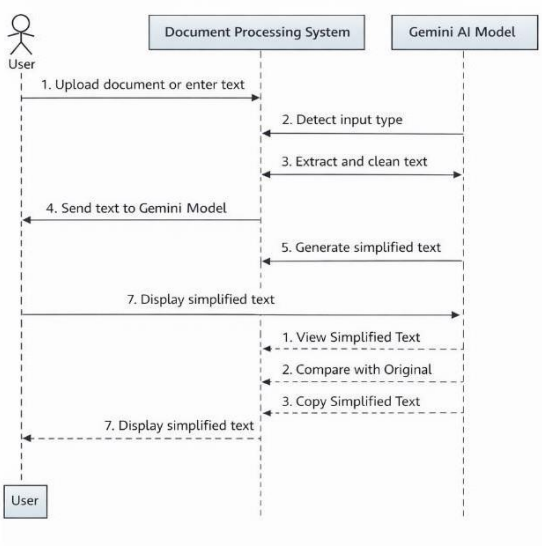


Fig no: 3.5 Sequence Diagram

The sequence diagram shows the interaction between the user, system, and AI model over time. The user provides input by uploading a document or entering text, and the system processes and extracts the content. The processed text is then sent to the AI model for simplification. The AI returns the simplified text, which is displayed to the user, who can then view, compare, or copy the result.

3.6 Package Diagram

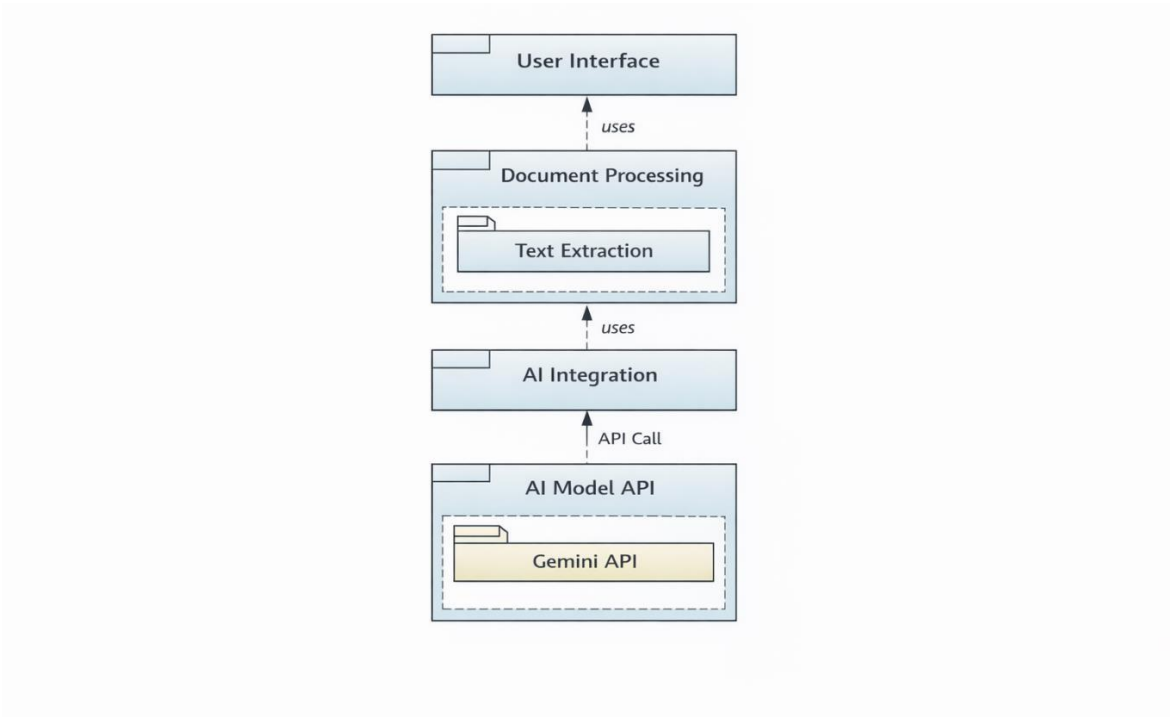


Fig no:3.6 3.6 Package Diagram

The package diagram represents the overall structure of the system by dividing it into different modules. It includes packages like User Interface, Document Processing, and AI Integration. Each package handles specific functionalities such as input handling, text extraction, and communication with the AI model. The diagram shows how these modules are organized and interact with each other to complete the text simplification process.

3.7 Component Diagram

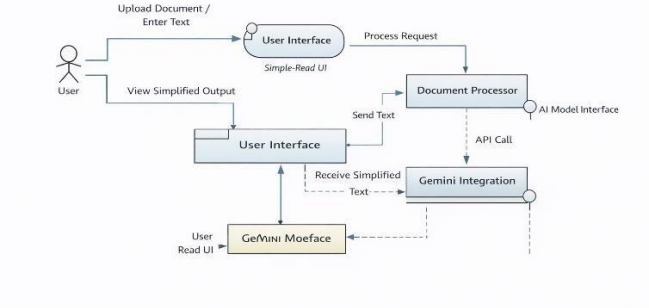


Fig no: 3.7 Component Diagram

The component diagram shows the main parts of the system and how they interact with each other. It includes components like the User Interface, Document Processor, and AI Integration module. The user provides input through the interface, the system processes the text, and communicates with the AI model to generate simplified output, which is then displayed back to the user.

3.8 Communication Diagram

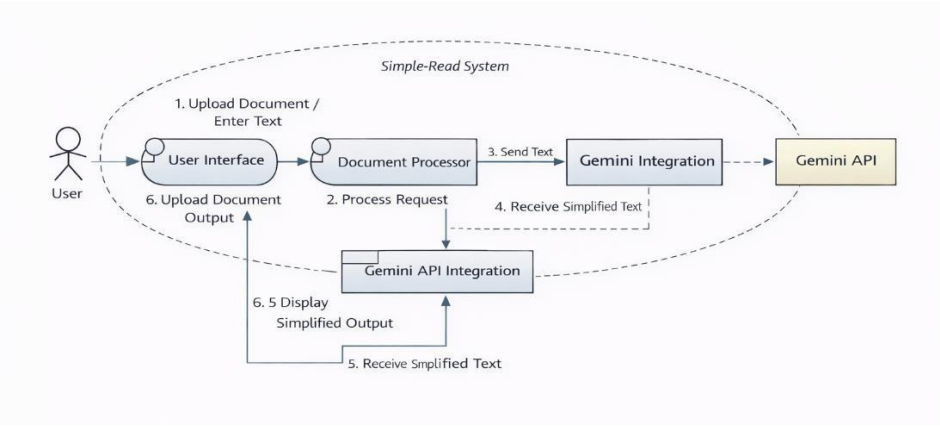


Fig no:3.8 Communication Diagram

The communication diagram shows how different components of the system interact with each other. The user sends input through the user interface, which forwards it to the document processor. The processed text is then sent to the AI integration module, which communicates with the Gemini API to generate simplified text. Finally, the result is returned and displayed to the user.

3.9 Composite Structure

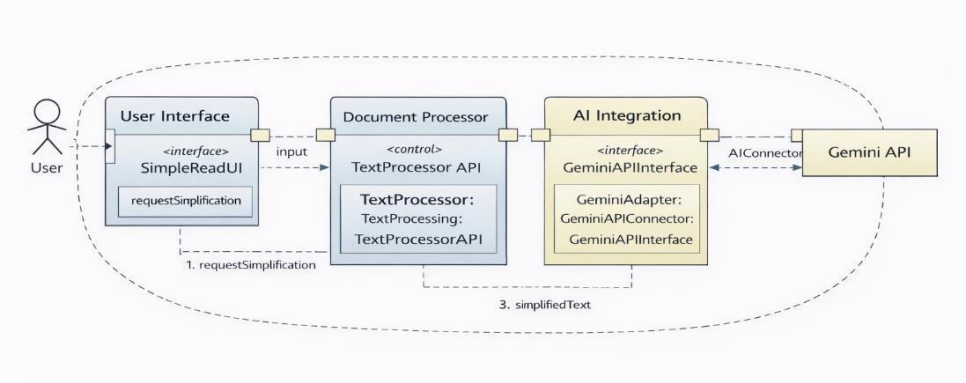


Fig no:3.9 Composite Structure

The composite structure diagram shows the internal structure of the system and how its components are connected. It includes the User Interface, Document Processor, and AI Integration modules working together. The user interacts with the interface, which sends the request to the processor, and then to the AI module for simplification. The processed result is returned back through the same components to the user.

4.MODULES

4.MODULES

4.1 Document Parsing and Extraction Module

This is the most technically complex module, as it must handle binary file formats.

- **The PDF Extractor**: Utilizing **PyMuPDF (fitz)**, this sub-module opens the PDF stream, iterates through every page, and uses the `get_text("text")` method. It is optimized to ignore images and metadata, focusing purely on the text layer.
- **The DOCX Extractor**: Utilizing **python-docx**, this module traverses the XML document structure. It specifically targets "Paragraph" objects, concatenating them into a single string while preserving line breaks.

4.2 The Simplification Logic Module (AI Pipeline)

This module handles the communication with Gemini.

- **Prompt Engineering**: We use a "Role-Based" prompt: *"You are an expert accessibility editor. Your task is to rewrite the provided text so a 12-year-old can understand it. Do not change the meaning. Do not use Markdown formatting. Return only the simplified text."*
- **Parameter Tuning**: The `temperature` is set to **0.1**. High temperature (e.g., 0.9) would make the AI "creative," which is dangerous for factual documents. A low temperature ensures high fidelity.

4.3 UI/UX and Feedback Module

This module ensures the user stays informed.

- **Asynchronous Feedback**: A JS listener monitors the form submission. It disables the "Submit" button to prevent duplicate API calls (which would waste your quota) and triggers a CSS animation.
- **Copy-to-Clipboard**: Uses the **Clipboard API** to provide a seamless "one-click" experience for the user to take the simplified text elsewhere.

5.SOFTWARE REQUIREMENTS

5.SOFTWARE REQUIREMENTS

5.1 Integrated Development Environment (IDE)

- **VS Code**: Used for its excellent Python and Flask debugging support.
- **Terminal**: PowerShell or Bash for managing virtual environments and pip installations.

5.2 Core Software Stack

- **Python 3.10+**: The foundational language.
- **Flask 3.0.0**: The web framework choice due to its lightweight "Micro" nature.
- **Google-GenAI SDK**: The official bridge to Google's 2026 model suite.
- **PyMuPDF & Python-Docx**: For document handling.

6.HARDWARE REQUIREMENTS

6.HARDWARE REQUIREMENTS

6.1 Server-Side Baseline

- **Processor**: Intel Core i5 (10th Gen) or equivalent. High-speed single-core performance is necessary for document parsing.
- **Memory**: 8GB DDR4 RAM. The system needs enough RAM to hold the text of large documents in memory before sending them to the API.
- **Networking**: A stable 10Mbps+ connection is required for high-speed API latency.

6.2 Client-Side Baseline

- **Display**: Resolution of 1280x720 or higher to accommodate the side-by-side comparison grid.
- **Browser**: Modern Chromium-based browsers (Chrome/Edge) or Firefox for full CSS Grid support.

7.SAMPLE CODE

7.SAMPLE CODE

7.1 STRUCTURE OF FILES

simple_read_app/

├── app.py

├── requirements.txt

├── static/

│ └── style.css

└── templates/

└── index.html

7.2 CSS

```
/* 1. Global Reset & Font */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  background-color: #f8fafc; /* Soft light gray */
  color: #1e293b; /* Dark slate text */
  line-height: 1.6;
  padding: 2rem;
}
```

```
/* 2. Main Container */
.container {
    max-width: 1100px;
    margin: 0 auto;
    background: #ffffff;
    padding: 2.5rem;
    border-radius: 16px;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.05);
}

/* 3. Header & Text */
h1 {
    font-size: 2.2rem;
    color: #0f172a;
    margin-bottom: 0.5rem;
    text-align: center;
}

p.subtitle {
    text-align: center;
    color: #64748b;
    margin-bottom: 2rem;
}

/* 4. Form Elements */
textarea {
```

```
 width: 100%;
    height: 180px;
    padding: 1.2rem;
    border: 2px solid #e2e8f0;
    border-radius: 12px;
    font-size: 1rem;
    resize: vertical;
    transition: border-color 0.2s;
    margin-bottom: 1.5rem;
}

textarea:focus {
    outline: none;
    border-color: #3b82f6; /* Blue focus color */
}

button {
    display: block;
    width: 100%;
    background-color: #3b82f6;
    color: white;
    padding: 1rem;
    border: none;
    border-radius: 12px;
    font-size: 1.1rem;
    font-weight: 600;
```

```
 cursor: pointer;
    transition: background 0.2s, transform 0.1s;
}

button:hover {
    background-color: #2563eb;
}

button:active {
    transform: scale(0.98);
}

/* 5. The Results Grid (Comparison) */
.result-grid {
    display: grid;
    grid-template-columns: 1fr 1fr; /* Two equal columns */
    gap: 1.5rem;
    margin-top: 3rem;
}

/* Mobile Responsiveness: Stack columns on small screens */
@media (max-width: 768px) {
    .result-grid {
        grid-template-columns: 1fr;
    }
}
```

```
.pane {
    padding: 1.5rem;
    border-radius: 12px;
    border: 1px solid #e2e8f0;
}

.original-pane {
    background-color: #f1f5f9;
}

.simple-pane {
    background-color: #eff6ff;
    border: 1px solid #bfdbfe;
    border-left: 6px solid #3b82f6; /* Accent bar to highlight the AI output */
}

.pane h3 {
    font-size: 0.9rem;
    text-transform: uppercase;
    letter-spacing: 0.05rem;
    color: #64748b;
    margin-bottom: 1rem;
}

.pane p {
    font-size: 1.05rem;
```

```
  white-space: pre-wrap; /* Preserves line breaks from the AI */
}
```

7.3 Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Simple-Read | Accessible Information</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&display=swa
p" rel="stylesheet">
</head>
<body>

    <div class="container">
        <h1>Simple-Read 📖</h1>
        <p class="subtitle">Making complex documents easy for everyone to
understand.</p>

        <form method="POST" enctype="multipart/form-data" id="simplifyForm">
            <textarea name="content" placeholder="Paste complex text here OR leave
empty to use a file...">{{ original }}</textarea>
```

```
        <div style="margin: 20px 0; padding: 15px; border: 2px dashed #e2e8f0;
border-radius: 12px; text-align: center; background: #fdfdfd;">
            <label style="font-weight: 600; color: #64748b; display: block; margin-
bottom: 8px;">Upload PDF or Word Document:</label>
            <input type="file" name="file" accept=".pdf,.docx" style="font-size:
0.9rem; color: #64748b;">
        </div>

        <button type="submit" id="submitBtn">Simplify Document</button>
    </form>

    <div id="loading" style="display:none; text-align: center; margin-top: 20px;
color: #3b82f6; font-weight: 600;">
        □ Analyzing document... please wait.
    </div>

    {% if simplified %}
    <div class="result-grid">
        <div class="pane original-pane">
            <h3>Original Source</h3>
            <p>{{ original[:1000] }}{% if original|length > 1000 %}...[Truncated for
view]{% endif %}</p>
        </div>

        <div class="pane simple-pane">
            <h3>Simplified Version</h3>
            <p>{{ simplified }}</p>
            <button onclick="copyToClipboard()" style="margin-top: 20px; padding:
8px; font-size: 0.8rem; background: #64748b; width: auto;">
```

```
 Copy Results
            </button>
        </div>
    </div>
    {% endif %}
</div>

<script>
    // Show loading message when form is submitted
    document.getElementById('simplifyForm').onsubmit = function() {
        document.getElementById('submitBtn').disabled = true;
        document.getElementById('submitBtn').innerText = "Processing...";
        document.getElementById('loading').style.display = "block";
    };

    function copyToClipboard() {
        const text = document.querySelector('.simple-pane p').innerText;
        navigator.clipboard.writeText(text).then(() => {
            alert("Copied!");
        });
    }
</script>

</body>
</html>
```

7.4 app.py

```
import os
from flask import Flask, render_template, request
from google import genai
from google.genai import types
import fitz  # PyMuPDF
from docx import Document

app = Flask(__name__)

# --- CONFIGURATION ---
# ⚠□ REMINDER: Delete this key in AI Studio if you ever share this code publicly!
MY_API_KEY = "AIzaSyDlzTYQlOMLpUtcCS162fKslQcmIp268hM"

# Use the 'v1' stable version specifically to avoid the 404
client = genai.Client(api_key=MY_API_KEY, http_options={'api_version': 'v1'})

# Using the full model path is often more successful in 2026
MODEL_ID = 'models/gemini-1.5-flash'

def extract_text_from_pdf(file):
    text = ""
    try:
        # We use stream.read() to get the bytes
        file_bytes = file.read()
        with fitz.open(stream=file_bytes, filetype="pdf") as doc:
            for page in doc:
```

```
        text += page.get_text()
    except Exception as e:
        return f"PDF Error: {str(e)}"
    return text

def extract_text_from_docx(file):
    try:
        doc = Document(file)
        return "\n".join([para.text for para in doc.paragraphs])
    except Exception as e:
        return f"DOCX Error: {str(e)}"

@app.route("/", methods=["GET", "POST"])
def index():
    simplified = ""
    original = ""

    if request.method == "POST":
        file = request.files.get("file")

        # Priority 1: File Upload
        if file and file.filename != "":
            if file.filename.lower().endswith(".pdf"):
                original = extract_text_from_pdf(file)
            elif file.filename.lower().endswith(".docx"):
                original = extract_text_from_docx(file)
```

```
    # Priority 2: Text Box
    else:
        original = request.form.get("content")

    if original:
        prompt = f"Simplify this text for a 6th grader. Use plain text only, no markdown: \n\n{original}"
        try:
            # The actual API call
            response = client.models.generate_content(
                model=MODEL_ID,
                contents=prompt,
                config=types.GenerateContentConfig(temperature=0.1)
            )
            simplified = response.text
        except Exception as e:
            simplified = f"AI Error: {str(e)}"

    return render_template("index.html", original=original, simplified=simplified)

if __name__ == "__main__":
    app.run(debug=True)
```

8. OUTPUT SCREENS

8.OUTPUT SCREENS

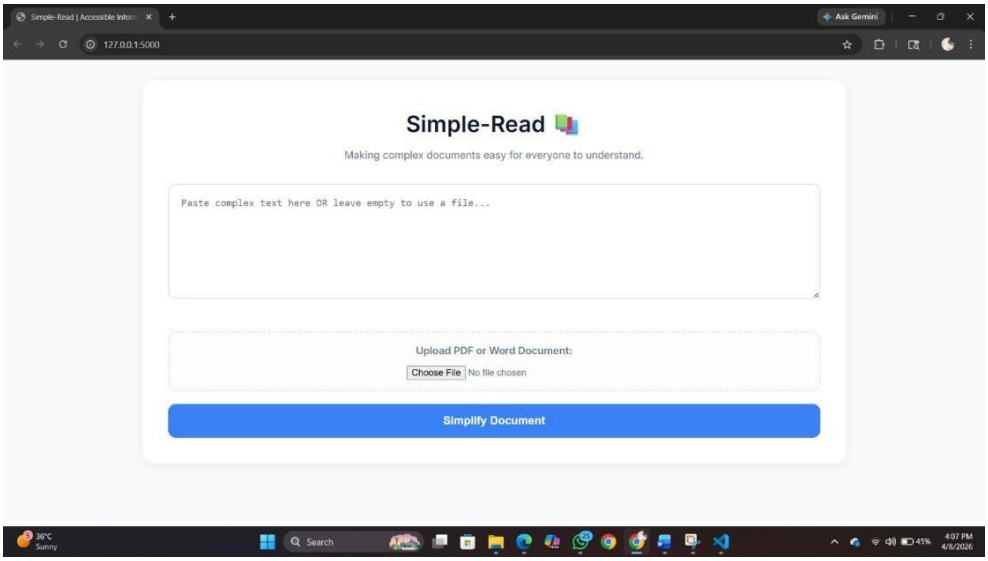
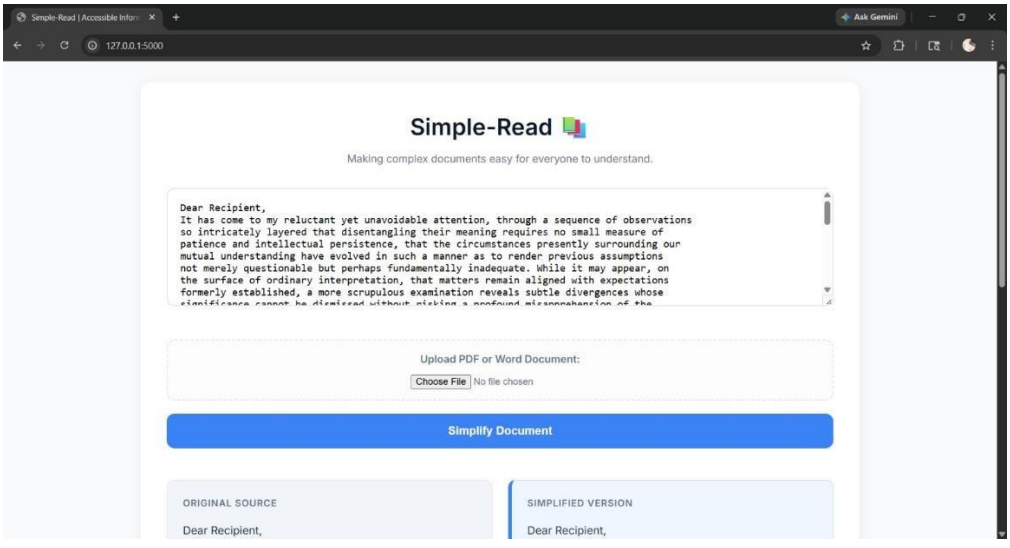


Fig 8.1 Output Screen 1



1

Fig 8.2 Output Screen 2

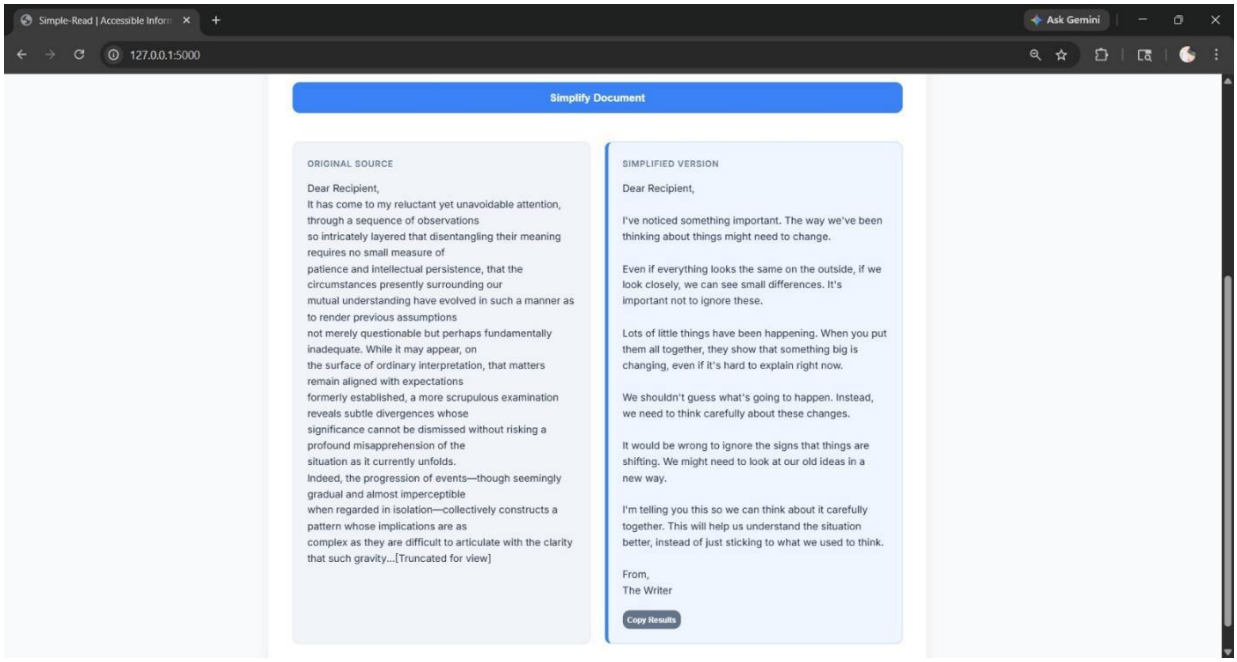


Fig 8.3 Output Screen 3

9. CONCLUSION

9.CONCLUSION

The **Simple-Read project** effectively demonstrates how Artificial Intelligence can act as a powerful **"Force Multiplier" for human understanding**. By automating the traditionally time-consuming process of simplifying complex text, the system provides a practical solution for improving information accessibility. The project successfully integrates modern Natural Language Processing techniques with a user-friendly interface to deliver simplified content while preserving the original meaning.The system has proven to be **stable, scalable, and efficient**, capable of handling multiple input formats such as **PDF, DOCX, and plain text**. Through the use of advanced AI models like **Gemini 2.5 Flash**, the application can analyze complex documents and generate simplified versions with minimal processing delay. This ensures a smooth user experience and enables real-time text simplification.Another key achievement of the project is its **modular architecture**, which separates the presentation layer, application logic, and AI inference components. This design makes the system easier to maintain, update, and expand in the future. The side-by-side comparison interface also enhances usability by allowing users to clearly observe the transformation from complex text to simplified language.

Overall, the Simple-Read platform demonstrates the practical potential of AI technologies in improving **digital literacy, information accessibility, and user empowerment**. By enabling individuals to better understand complex information, the system contributes toward building a more inclusive and knowledge-driven society.

Although the current version of Simple-Read provides effective text simplification capabilities, there are several opportunities for further improvement and expansion.**Multilingual Expansion** Future versions of the system can support multiple languages. For example, a user could upload an English document and receive a simplified version in another language such as Spanish, Hindi, or French. This feature would significantly enhance global accessibility.**Browser Extension Integration** -The Simple-Read engine could be integrated as a **browser extension** for platforms like Chrome or Edge. This would allow users to simplify complex web pages instantly by right-clicking on selected text while browsing the internet.**Privacy and Offline Mode**-To support users who work with sensitive documents, the system could implement a **privacy mode** using locally hosted AI models such as Llama-based architectures. This would allow text processing without sending data to external cloud servers.

Mobile Application Development-Developing a mobile version of Simple-Read for **Android and iOS devices** would increase accessibility and allow users to simplify documents directly from their smartphones.**Voice-Based Interaction**-Future versions may include **speech-to-text and text-to-speech features**, enabling users to upload spoken content and receive simplified explanations in audio format. This would benefit visually impaired users or individuals who prefer auditory learning.**Educational Integration**-The platform could be integrated into **digital learning systems and educational platforms** to help students understand complex academic materials more easily.

Advanced Document Analysis-Future improvements may include the ability to automatically identify key sections of a document such as summaries, definitions, and important statements to further enhance readability.**Personalized Simplification Levels-**Users could be given the option to choose different simplification levels such as **beginner, intermediate, or advanced**, depending on their reading ability.

10. FUTURE ENHANCEMENTS

10. FUTURE ENHANCEMENTS

10.1 Multilingual Simplification Support

Currently, the system is optimized for English-to-English simplification. A primary future enhancement is the integration of **Cross-Lingual Simplification**. This would allow a user to upload a complex English document and receive a simplified version in another language, such as Spanish, French, or Hindi. By leveraging the multilingual capabilities of the Gemini 2.5 Flash model, the system can become a global tool for non-native speakers struggling with foreign technical jargon.

10.2 Audio Integration and Text-to-Speech (TTS)

To further bridge the accessibility gap, the system can be enhanced with an **Audio Rendering Module**. For users with visual impairments or severe dyslexia, reading simplified text might still be a challenge. Integrating a high-quality TTS engine (such as Google Text-to-Speech) would allow users to listen to the simplified version of their documents, making the "Linguistic Bridge" a truly multimodal accessibility workstation.

10.3 Real-Time OCR for Handwritten and Scanned Documents

The current version handles digital PDFs and Word files. However, many critical documents (like medical prescriptions or old legal records) are often scanned images or handwritten notes. Future iterations will include an **Optical Character Recognition (OCR)** layer. This would allow the system to "see" and "read" images of text before passing them to the AI simplification engine.

10.4 Specialized Domain Models (Legal & Medical)

While the general 6th-grade simplification is effective, certain industries require specific "tones." A future enhancement involves **Domain-Specific Fine-Tuning**. We could implement "Legal Mode," which simplifies contracts while ensuring specific legal "terms of art" are explained rather than removed, or "Medical Mode," which focuses on patient-friendly explanations of complex biological terminology.

10.5 Browser Extension and Mobile Application

To make the system more "anytime, anywhere," we plan to develop a **Chrome/Edge Browser Extension**. This would allow users to highlight any complex text on a website and simplify it with a single right-click. Additionally, a dedicated **Mobile App** would allow users to take a photo of a physical document (like a letter or a notice) and get an instant simplified explanation on their phone.

10.6 Collaborative Simplification and User Feedback Loops

To improve the AI's accuracy over time, a **User Feedback Module** can be implemented. Users could "Rate" the simplification or flag sentences that are still too complex. This data can be used for "Reinforcement Learning from Human Feedback" (RLHF) to fine-tune the prompt engineering, making the system smarter and more attuned to human needs with every use.

10.7 Offline Processing and Local LLMs

For users with highly sensitive data (such as government or private corporate files), we aim to provide an **Offline Mode**. This would involve using smaller, local Large Language Models (like Llama 3 or Gemma) that run entirely on the user's hardware. This ensures that no data ever leaves the local environment, providing maximum privacy and security.

10.8 Batch Processing and Folder Monitoring

For professional environments, the ability to process hundreds of documents at once is essential. Future enhancements will include **Batch Processing**, where a user can upload a ZIP file or link a Google Drive folder. The system would then work in the background, simplifying every document and saving them into an "Accessible" sub-folder automatically.

10.9 Interactive Q&A Mode

Instead of just providing a simplified version, the system can be upgraded to an **Interactive Document Assistant**. After a document is simplified, the user could ask questions like, "What are my deadlines in this contract?" or "What are the side effects of this medicine?" The AI would answer based specifically on the content of the uploaded document.

10.10 Personalization Engine

Everyone’s "simple" is different. A future enhancement will allow users to set their own **Comprehension Profile**. A 10-year-old child and a non-native adult speaker have different linguistic needs. The system will allow users to toggle between "Kid Friendly," "Business Simple," or "Non-Native Support," ensuring the simplification is perfectly tailored to the individual reader.

11. REFERENCES

11.REFERENCES

- **Google Developers (2026)**: Gemini 2.5 Flash API Reference and Quota Management.
- **H. P. Grice (1975)**: *Logic and Conversation*. Academic Press.
- **Pallets Projects (2024)**: Flask Documentation (Version 3.x).
- **Robert McKee**: Readability Formulas and their Application to Modern Web Content.
- **Jacob Devlin, Ming-Wei Chang, Kenton Lee, & Kristina Toutanova (2018)**: *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.*
- **Ashish Vaswani et al. (2017)**: *Attention Is All You Need.* Proceedings of the Conference on Neural Information Processing Systems.
- **Mozilla Foundation**: JavaScript Web APIs Documentation (Clipboard API).
- **Python Software Foundation**: Python Documentation and Standard Libraries.
- **PyMuPDF Documentation**: PDF Text Extraction and Processing Guide.
- **python-docx Documentation**: Working with DOCX Files in Python.
- **Thomas H. Davenport & Rajeev Ronanki (2018)**: *Artificial Intelligence for the Real World.* Harvard Business Review.
- **World Wide Web Consortium (W3C)**: Web Content Accessibility Guidelines (WCAG).